

Wallet & P2P Transfer - design and reasoning

Paytm PML R2 | Python 3.12 / FastAPI / PostgreSQL | Synthetic assessment funds

Data model. users stores UUID identities and hashes of random bearer tokens. wallets has one row per user (UNIQUE user_id), BIGINT current/opening balances, and a nonnegative check. transfers stores immutable final receipts, including declines. idempotency_keys has primary key (user_id, key), normalized request JSON, and a deferred foreign key to its result. domain_events persists committed business events. Values are strict integer paise, capped at JavaScript's exact integer limit; booleans, decimals, strings, and self-transfers are rejected.

Simplest correct transfer. One READ COMMITTED transaction reserves the key, locks both wallets with two SELECT ... FOR UPDATE statements in ascending UUID order, checks ownership and funds, conditionally debits, credits, writes the receipt/events, and commits. Both balance changes commit or neither does; overlapping debits serialize. Both A-to-B and B-to-A lock the lower ID first. Reservation occurs in a separate table so transfer foreign keys cannot take wallet locks out of order. A destination overflow is also declined before any debit. Wallet get-or-create uses INSERT ON CONFLICT DO NOTHING and a separate fresh-snapshot SELECT.

Alternatives rejected. Serializable isolation adds broad conflict retries; optimistic version checks add a retry loop for both wallets. Redis/process locks add lease or multi-instance failure modes and cannot replace database atomicity. A global lock blocks unrelated transfers. Queues and sagas add pending states and compensation where one database transaction suffices. A full double-entry ledger is a sensible production extension; immutable receipts and reconciliation suffice for this assessment's bounded model.

Idempotency and failure. PostgreSQL uniqueness, not application memory, arbitrates same-key races. Reservation, debit, credit, receipt, and events share one transaction; the deferred FK prevents an orphan key from committing. Replays compare the normalized body and return the stored receipt; different valid bodies return 409. Declines stay declined even after funding. Validation/authorization failures roll back reservations. A crash before commit rolls back; a lost response after commit is recovered by retrying the same key. Transport errors can have an unknown commit outcome.

Consistency and operations. All reads/writes use one primary; no stale balance cache or offline acceptance. DB outages and bounded lock/pool waits yield 503; availability is consciously sacrificed to preserve money correctness. The non-root, multi-stage Docker image has a HEALTHCHECK; Compose runs it with PostgreSQL. JSON logs carry correlation IDs; a public sanitized tail/stream and durable event endpoint support inspection. Prometheus counters/histograms expose rate, p99, 5xx, declines, and replays. Process metrics/tails reset on restart; committed event rows survive.

Verification and orchestration. The standard-library burst script checks wallet races, same-key storms, opposite-direction contention, receipt reconciliation, changed-body conflicts, and competing overdrafts. The Docker/CI gate also inspects image settings, injects pre-commit and post-commit failures, and checks replay after an app restart. evidence/verification.md identifies executed versus pending gates; public URLs must be filled only after deployment and a live burst.

AI disclosure and cost. Human-directed: the supplied brief and request to build the backend. AI-decided/generated: stack, transaction and schema choices, tests, observability, and packaging. Human independent review/deployment is not claimed. The Render Blueprint selects Free web service plus Free managed PostgreSQL: intended cost INR 0, with no paid upgrade. Account eligibility is unverified; Render's free DB expires after 30 days and its free web service sleeps after idle. Publishing remains pending account connections. This is an assessment, not production payment infrastructure.

References: [PostgreSQL locking](#), [INSERT conflict semantics](#), [Render free-tier limits](#). Checked September 9, 2026.